

Topic 4.5

4.5: DynamicIntArray Example

Ordinary arrays (both automatic and dynamically-allocated) have several shortcomings:

- The size of automatic arrays must be known at compile time.
- Dynamic arrays have run-time determinable size, but correct allocation must be verified each time the array is allocated.
- Out-of-range subscripts are not caught.
- Array elements are not initialized (except for static arrays).

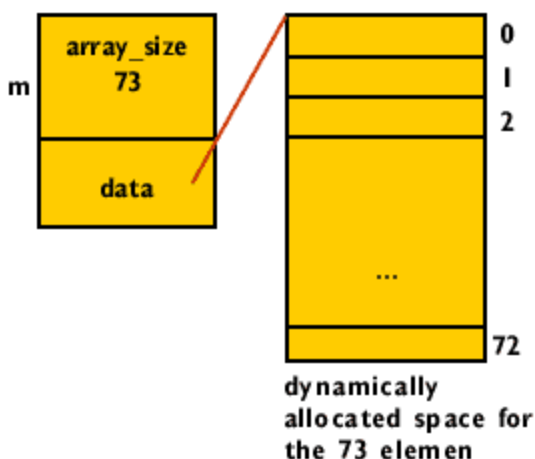
In the following example, we define a class *DynamicIntArray* that will allow us to dynamically create arrays with the following properties:

- The constructor checks for dynamic allocation success or failure.
- The constructor initializes all array locations to 0.
- To access the elements of the array, we overload the subscript operator `[]`. This allows us to verify that each access to an element of the array uses an in-range subscript. If not, the program terminates immediately.

To declare a variable of this class type, we use a declaration such as:

```
DynamicIntArray m(73);
```

where *73* is the size of the array we want. The value provided does *not* need to be known at compile time. It may be a variable, formal parameter to this function, etc. The object *m* looks like this:



The *array_size* and *data* fields are private because we don't want the user messing with them. However, we need a way to access the actual locations in the array. We do this by overloading *operator[]*, which allows us to write such code as:

```
m[2] = m[0] + m[1];
```

Notice that we are using subscripts, even though *m itself is not an array!* This is one of the real benefits of overloading, especially the *operator[]*.

Our definition of *operator[]* that applies to *DynamicIntArray* objects does two things:

m[i]

first verifies that *i* is a legal subscript (between 0 and 72). If so, it returns a *reference* to the *ith* element of the dynamically allocated memory.

Note that we have prohibited non-class members from directly accessing the *data* pointer (because they could change it, deallocate the space it points to, etc.). But we *do* allow access to the individual elements of the dynamic memory by handing back references to the appropriate locations.

My implementation of dynamic arrays includes an internal name for each array. This allows us to view the output of the driver program more meaningfully, as we can see each array as it is constructed, copied, and destructed. The source code for this example is among the files included in the zipped file that should have downloaded. You will find it beneficial to compile it and execute, watching the results.

```
// MODULE      dynarr1d.cpp
// PROGRAMMER: Gary J. Blair
// DATE:       4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    dynarr1d.cpp
// PURPOSE:
//   Driver for dynarr1 exercise.
//   Illustrates:
//     dynamic integer arrays with
//     subscript checking.
//     new, copy constructor,
//     overloading [], destructors.

#include <iostream>
#include <stdlib.h>
#include "dynarr1.h"

using namespace std;

void create_array( size_t size );
void print_array( DynamicIntArray a );

int main(void)
{
    size_t size;

    cout << "Enter the array size ";
    cin >> size;
    create_array(size);
    cout << "main terminating.\n";

    return 0;
}

void create_array( size_t size )
{
    DynamicIntArray x(size,"x"); // Dynamic
                                // array.
```

```

long            i;                // Loop
                                // variable.
DynamicIntArray *p;  // Ptr to
                    // DynamicIntArray.

cout << "The size of x is " << x.size() << endl;

x[0] = 1;
x[1] = 1;
for (i=2; i<x.size(); ++i)
    x[i] = x[i-1] + x[i-2];
print_array(x);

p = new DynamicIntArray(3,"*p");
cout << "The size of *p is " << p->size()
    << endl;
(*p)[0] = 1996;
(*p)[1] = 5;
(*p)[2] = 30;
print_array(*p);
*p = x;
cout << "The size of *p is now " << p->size()
    << endl;
print_array(*p);
delete p;

cout << "create_array returning.\n";
}

void print_array( DynamicIntArray a )
{
    size_t  i;  // Loop variable.

    for (i=0; i<a.size(); ++i)
        cout << "a[" << i << "] = " << a[i] << endl;
    cout << "print_array returning.\n";
}

```

Output from dynarr1d.cpp:

```

Enter the array size 5
constructor called for x
The size of x is 5
Copy constructor called to copy x
a[0] = 1
a[1] = 1
a[2] = 2
a[3] = 3
a[4] = 5
print_array returning.
Destructor called for copy of x
constructor called for *p
The size of *p is 3
Copy constructor called to copy *p
a[0] = 1996
a[1] = 5
a[2] = 30
print_array returning.
Destructor called for copy of *p
operator= called to copy x to *p
The size of *p is now 5
Copy constructor called to copy *p
a[0] = 1
a[1] = 1

```

```

a[2] = 2
a[3] = 3
a[4] = 5
print_array returning.
Destructor called for copy of *p
Destructor called for *p
create_array returning.
Destructor called for x
main terminating.

```

```

// MODULE      dynarr1.h
// PROGRAMMER: Gary J. Blair
// DATE:       4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    dynarr1d.cpp, dynarr1.cpp, dynarr1.h
// PURPOSE:
//   Declarations for class DynamicIntArray.
//   Illustrate: //      dynamic integer arrays with
//               subscript checking.
//               new, copy constructor,
//               overloading [], destructors.

#include <iostream>
#include <stdlib.h>

class DynamicIntArray
{
public:

    // Constructor.

    DynamicIntArray( size_t size,
                    const char * const name);

    // Copy Constructor.

    DynamicIntArray( const DynamicIntArray& rhs );

    // Destructor.

    ~DynamicIntArray(void);

    // Assignment operator=.

    DynamicIntArray& operator =
        DynamicIntArray& rhs);

    // Accessor function which returns
    // a reference to an element of
    // the dynamically-allocated memory.

    int& operator[]( long subscript );

    // Inquire the size of the array.

    size_t size(void);

private:
    size_t array_size;    // Size of the
                        // dynamic array.
    char array_name[80]; // For educational
                        // purposes only.
    int *data;           // Pointer to
                        // dynamic memory
                        // for the array.

```

```

// MODULE      dynarr1.cpp
// PROGRAMMER: Gary J. Blair
// DATE:       4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    dynarr1d.cpp, dynarr1.cpp, dynarr1.h
// PURPOSE:
//   Implimentation for class DynamicIntArray.
//   Illustrate: //      dynamic integer arrays with
//               subscript checking.
//               new, copy constructor, overloading [],
//               destructors.

#include        <iostream>
#include        <stdlib.h>
#include        <string.h>
#include        "dynarr1.h"

    // Constructor.

DynamicIntArray::DynamicIntArray
( size_t size, const char * const name )
{
    size_t i; // Loop variable.

    cout << "constructor called for " << name
        << endl;
    data = new int[size];
    if (! data)
    {
        cerr << "*** Memory allocation failed.\n";
        exit(1);
    }
    array_size = size;
    for (i=0; i<array_size; ++i)
        data[i] = 0;
    strcpy(array_name,name);
}

    // Destructor.

DynamicIntArray::~DynamicIntArray(void)
{
    delete data;
    array_size = 0;
    cout << "Destructor called for " << array_name
        << endl;
}

    // Copy Constructor.

DynamicIntArray::DynamicIntArray
( const DynamicIntArray& rhs )
{
    size_t i; // Loop variable.

    cout << "Copy constructor called to copy "
        << rhs.array_name << endl;

    data = new int[rhs.array_size];
    if (! data)
    {
        cerr << "*** Memory allocation failed.\n";
        exit(1);
    }
    array_size = rhs.array_size;
    for (i=0; i<array_size; ++i)
        data[i] = rhs.data[i];
    strcpy(array_name,"copy of ");
}

```

```

        strcat(array_name, rhs.array_name);
    }

    // Assignment operator=.

DynamicIntArray& DynamicIntArray::operator
= (DynamicIntArray& rhs)
{
    size_t i;    // Loop variable.

    cout << "operator= called to copy "
         << rhs.array_name << " to " << array_name
         << endl;

    if (this == &rhs)
        return *this;
    if (array_size != rhs.array_size)
    {
        delete data;    // Deallocate current space.
        data = new int[rhs.array_size];
        if (! data)
        {
            cerr << "*** Memory allocation failed.\n";
            exit(1);
        }
        array_size = rhs.array_size;
    }
    for (i=0; i<array_size; ++i)
        data[i] = rhs.data[i];
    return *this;
}

    // Accessor function which returns a reference
    // to an element of
    // the dynamically-allocated memory.

int& DynamicIntArray::operator[]( long subscript )
{
    if (subscript < 0 || subscript >
        (array_size - 1))
    {
        cerr << "*** Out of range subscript = "
             << subscript << endl;
        exit(1);
    }
    return data[subscript];
}

    // Inquire the size of the array.

size_t DynamicIntArray::size(void)
{
    return array_size;
}

```

Topic 4.6 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-4-dot-6>)